Estructura de datos Union-Find

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

3 de marzo de 2026

Union-Find Data Structure

Estudio de la estructura de datos Union-Find y sus aplicaciones.

Universidad Pontificia Bolivariana

Índice

1. El problema de componentes conexas / Conjuntos disjuntos **2**
1.1. Aplicaciones prácticas 2

2. API y representación **3**

3. Implementación de la estructura Union-Find **3**
3.1. Búsqueda rápida: QuickFind 3
3.2. Representación por árboles: QuickUnion 4
3.3. Unión ponderada: WeightedQuickUnion 5
3.4. Comparación de implementaciones 5

4. Optimizaciones y Análisis de Eficiencia **6**
4.1. Compresión de Caminos 6
4.2. Análisis amortizado 7

5. Ejercicios **8**

1. El problema de componentes conexas / Conjuntos disjuntos

El problema de componentes conexas (o conjuntos disjuntos) se centra en determinar la conectividad entre diferentes nodos en un grafo o red. Dos nodos se consideran conectados si existe un camino entre ellos. Esta relación de conectividad es una relación de equivalencia, lo que significa que cumple con las siguientes propiedades:

- **Reflexiva:** Todo nodo está conectado consigo mismo.
- **Simétrica:** Si el nodo A está conectado al nodo B, entonces el nodo B está conectado al nodo A.
- **Transitiva:** Si el nodo A está conectado al nodo B, y el nodo B está conectado al nodo C, entonces el nodo A está conectado al nodo C.

El objetivo principal es, dado un conjunto de nodos y un conjunto de conexiones entre ellos, poder determinar eficientemente si dos nodos dados están conectados y, en caso de que no lo estén, unirlos.

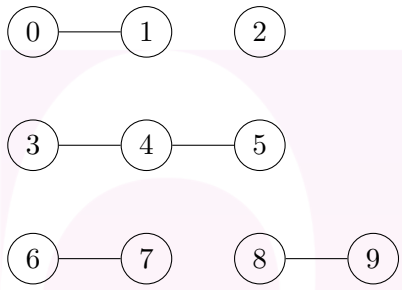

Figura 1: Un grafo con 10 nodos y 5 componentes conexas: {0, 1}, {2}, {3, 4, 5}, {6, 7}, {8, 9}.

1.1. Aplicaciones prácticas

La estructura Union-Find tiene numerosas aplicaciones en ciencias de la computación y problemas del mundo real:

- **Conectividad en redes:** Determinar si dos computadoras están conectadas en una red, directa o indirectamente.
- **Algoritmo de Kruskal:** Encontrar el árbol de expansión mínima de un grafo ponderado.
- **Percolación:** Modelar la propagación de fluidos a través de materiales porosos o la conectividad en sistemas físicos.
- **Procesamiento de imágenes:** Identificar componentes conexas en imágenes digitales para segmentación.
- **Equivalencia de variables:** En compiladores, determinar qué variables son equivalentes.
- **Detección de ciclos:** Determinar si agregar una arista a un grafo crea un ciclo.

2. API y representación

La estructura Union-Find gestiona un universo de n elementos distintos, inicialmente cada uno en su propio conjunto disjunto. Convencionalmente, los elementos se representan con números naturales de 0 a $n-1$. Para gestionar estos conjuntos, el API básico consta de las siguientes operaciones:

- **find(p):** Determina el **representante** del conjunto al que pertenece el elemento p. Dos elementos p y q están en el mismo conjunto si y solo si ‘find(p) == find(q)‘.
- **connected(p, q):** Verifica si los elementos p y q están en el mismo conjunto. Es una operación de conveniencia equivalente a comparar los resultados de ‘find(p)‘ y ‘find(q)‘.
- **union(p, q):** Une los conjuntos que contienen a los elementos p y q. Si ya estaban en el mismo conjunto, no hace nada.
- **count():** Devuelve el número de conjuntos disjuntos.

3. Implementación de la estructura Union-Find

3.1. Búsqueda rápida: QuickFind

La implementación **QuickFind** utiliza un arreglo ‘id[]‘ de tamaño n, donde ‘id[i]‘ es el identificador del componente al que pertenece el elemento i. Inicialmente, cada elemento está en su propio conjunto, por lo que ‘id[i] = i‘ para todo i.

Algorithm 1 QuickFind

```
Estructura: Arreglo ‘id[]‘ de tamaño n.
Inicialización: ‘id[i] = i‘ para i = 0, ..., n − 1.
procedure FIND(p)
    return ‘id[p]‘
end procedure
procedure UNION(p, q)
    p_id ← FIND(p)
    q_id ← FIND(q)
    if p_id ≠ q_id then
        for i ← 0 to n − 1 do
            if id[i] = p_id then
                id[i] ← q_id
            end if
        end for
    end if
end procedure
```

QuickFind recibe su nombre porque ofrece una operación `find()` muy rápida ($O(1)$): solo requiere un acceso al arreglo. Sin embargo, la operación `union()` es lenta ($O(n)$) porque debe recorrer todo el arreglo para actualizar todos los elementos del componente. Esto la hace ineficiente para problemas con un gran número de uniones.

Ejemplo: Considere $n = 5$ con elementos $\{0, 1, 2, 3, 4\}$. Inicialmente, `id[]` $= [0, 1, 2, 3, 4]$. Después de `union(0, 1)`, todos los elementos con id 0 (solo el 0) cambian a id 1: `id[]` $= [1, 1, 2, 3, 4]$.

Luego, `union(2, 3)` da `id[]` $= [1, 1, 3, 3, 4]$. Finalmente, `union(1, 3)` debe cambiar todos los elementos con id 1 (elementos 0 y 1) a id 3: `id[]` $= [3, 3, 3, 3, 4]$. Esta última operación requirió recorrer todo el arreglo.

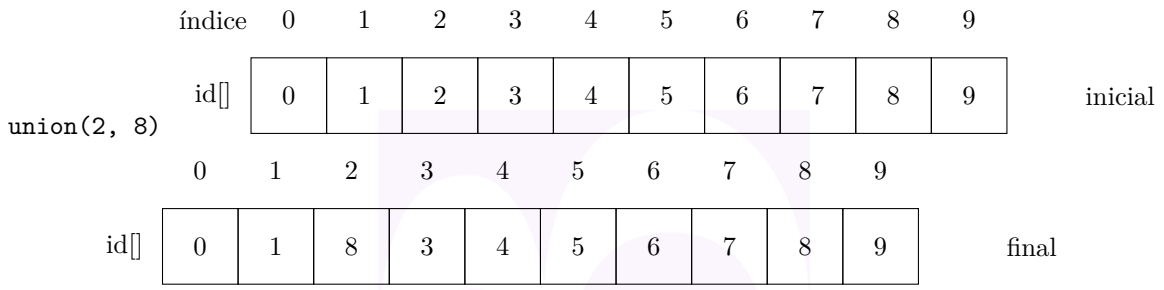

Figura 2: En QuickFind, para unir 2 y 8, se cambia el id de todos los elementos con id 2 al id 8. En este caso, solo se cambia ‘id[2]‘.

3.2. Representación por árboles: QuickUnion

La implementación **QuickUnion** utiliza un arreglo ‘parent[]‘ de tamaño n, donde ‘parent[i]‘ es el padre del elemento i. Cada conjunto se representa como un árbol, y la raíz del árbol es el representante del conjunto. Inicialmente, cada elemento es su propio padre (‘parent[i] = i‘).

Algorithm 2 QuickUnion

```
Estructura: Arreglo ‘parent[]‘ de tamaño n.
Inicialización: ‘parent[i] = i‘ para i = 0, ..., n − 1.
procedure FIND(p)
    while p ≠ parent[p] do
        p ← parent[p]
    end while
    return p
end procedure
procedure UNION(p, q)
    root_p ← FIND(p)
    root_q ← FIND(q)
    if root_p ≠ root_q then
        parent[root_p] ← root_q
    end if
end procedure
```

QuickUnion recibe su nombre porque la operación `union()` es conceptualmente rápida: solo requiere cambiar un apuntador una vez encontradas las raíces. Sin embargo, depende del tiempo de la operación `find` y esta a su vez puede ser lenta si los árboles se vuelven muy altos. En el peor caso, un árbol puede degenerar en una lista enlazada, haciendo que `find` tome tiempo $O(n)$. Aunque la complejidad del peor caso no mejora respecto a QuickFind, esta estructura arbórea es la base para optimizaciones mucho más eficientes.

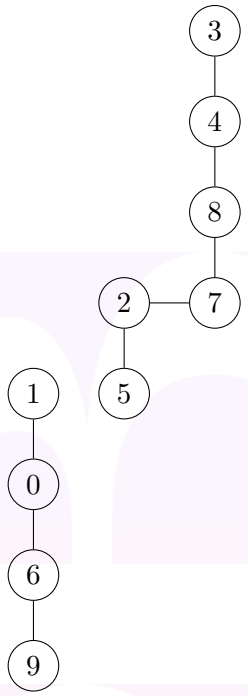

Figura 3: Peor caso en QuickUnion. Uniones como ‘union(9,6)‘, ‘union(6,0)‘, ‘union(0,1)‘ pueden crear árboles altos y delgados, llevando la complejidad de ‘find‘ a $O(n)$.

3.3. Unión ponderada: WeightedQuickUnion

Esta es una mejora de QuickUnion que evita la formación de árboles altos. Se usa un arreglo adicional ‘size[]‘, donde ‘size[i]‘ almacena el número de nodos en el árbol con raíz i. En una operación ‘union‘, siempre se enlaza la raíz del árbol más pequeño a la raíz del árbol más grande.

Esta simple heurística asegura que la altura de los árboles sea como máximo $O(\log n)$. Como resultado, las operaciones ‘find‘ y ‘union‘ tienen una complejidad de $O(\log n)$.

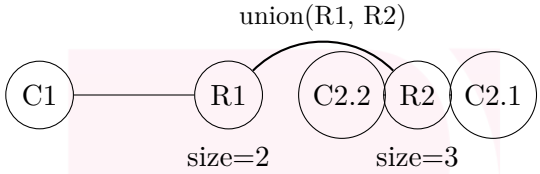

Figura 4: En WeightedQuickUnion, al unir dos árboles, el más pequeño (raíz R1, tamaño 2) se enlaza al más grande (raíz R2, tamaño 3), manteniendo el árbol resultante más chato.

3.4. Comparación de implementaciones

La siguiente tabla resume las complejidades temporales de las tres implementaciones básicas:

Algorithm 3 WeightedQuickUnion

```
Estructura: Arreglos 'parent[]' y 'size[]' de tamaño n.
Inicialización: 'parent[i] = i', 'size[i] = 1' para i = 0, ..., n − 1.
procedure FIND(p)
    while p ≠ parent[p] do
        p ← parent[p]
    end while
    return p
end procedure
procedure UNION(p, q)
    root_p ← FIND(p)
    root_q ← FIND(q)
    if root_p ≠ root_q then
        if size[root_p] < size[root_q] then
            parent[root_p] ← root_q
            size[root_q] ← size[root_q] + size[root_p]
        else
            parent[root_q] ← root_p
            size[root_p] ← size[root_p] + size[root_q]
        end if
    end if
end procedure
```

Implementación	find()	union()	Observaciones
QuickFind	$O(1)$	$O(n)$	Ineficiente para muchas uniones
QuickUnion	$O(n)$	$O(n)$	Peor caso: árbol degenerado
WeightedQuickUnion	$O(\log n)$	$O(\log n)$	Altura acotada por $\log n$
WQU + Path Compression	$O(\alpha(n))^*$	$O(\alpha(n))^*$	Prácticamente constante

Cuadro 1: Complejidades de las implementaciones de Union-Find. *Costo amortizado.

4. Optimizaciones y Análisis de Eficiencia

4.1. Compresión de Caminos

La compresión de caminos es una optimización adicional para QuickUnion (y sus variantes) que aplana la estructura del árbol durante las operaciones 'find'. Cuando se busca la raíz de un nodo 'p', se hace que todos los nodos en el camino desde 'p' hasta la raíz apunten directamente a la raíz.

La implementación de la compresión de caminos puede hacerse de dos formas principales: con un segundo paso o con una sola pasada. La versión más simple modifica el método 'find' para actualizar los apuntadores durante la búsqueda:

Una variante más simple es la **compresión de caminos por halving**, que solo requiere una pasada y hace que cada nodo apunte a su abuelo durante la búsqueda:

Con la unión ponderada y la compresión de caminos, la eficiencia de la estructura de datos mejora drásticamente.

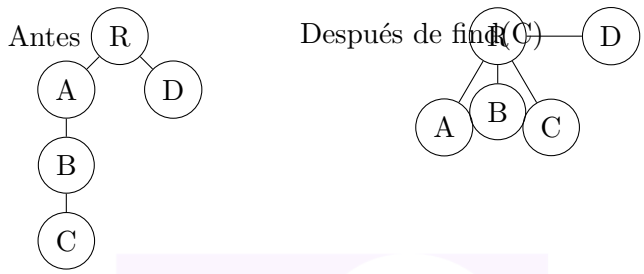

Figura 5: Efecto de la compresión de caminos al ejecutar ‘find(C)‘. Los nodos A, B y C ahora apuntan directamente a la raíz R.

Algorithm 4 find con compresión de caminos (two-pass)

```
procedure FIND(p)
    root ← p
    while root ≠ parent[root] do
        root ← parent[root]
    end while
    while p ≠ root do
        next ← parent[p]
        parent[p] ← root
        p ← next
    end while
    return root
end procedure
```

4.2. Análisis amortizado

El **análisis amortizado** se usa para analizar secuencias de operaciones. En lugar de acotar el costo de la peor operación individual, se promedia el costo sobre una secuencia de operaciones. Este tipo de análisis es apropiado cuando operaciones ocasionalmente costosas son compensadas por muchas operaciones baratas.

Para Union-Find con unión ponderada y compresión de caminos, se ha demostrado (Tarjan, 1975) que cualquier secuencia de m operaciones sobre n elementos toma un tiempo casi lineal, del orden de $O(m \cdot \alpha(n))$, donde $\alpha(n)$ es la función inversa de Ackermann.

La función $\alpha(n)$ crece de forma extremadamente lenta:

- $\alpha(1) = 0$

Algorithm 5 find con path halving (one-pass)

```
procedure FIND(p)
    while p ≠ parent[p] do
        parent[p] ← parent[parent[p]]        ▷ Apunta al abuelo
        p ← parent[p]
    end while
    return p
end procedure
```

- $\alpha(2^{16}) = 3$
- $\alpha(2^{65536}) = 4$
- Para $n < 2^{2^{65536}}$, $\alpha(n) \leq 5$

Como el número 2^{65536} es astronómicamente mayor que el número de átomos en el universo observable ($\approx 10^{80}$), para cualquier aplicación práctica, $\alpha(n) \leq 5$. Por lo tanto, el costo amortizado por operación es prácticamente constante: $O(\alpha(n)) \approx O(1)$ en la práctica.

5. Ejercicios

1. Considere un conjunto de 10 elementos $0, 1, \ldots, 9$. Muestre el contenido del arreglo `parent[]` después de la siguiente secuencia de uniones usando:

   (a) QuickUnion

   (b) WeightedQuickUnion

   Secuencia: `union(4, 3)`, `union(3, 8)`, `union(6, 5)`, `union(9, 4)`, `union(2, 1)`, `union(5, 0)`, `union(7, 2)`, `union(6, 1)`, `union(7, 3)`.

2. ¿Cuál es la altura máxima de un árbol en una estructura WeightedQuickUnion con N elementos? Explique su razonamiento y demuestre por qué esta cota es ajustada.

3. Describa una secuencia de $N - 1$ operaciones de unión sobre N elementos que resulte en un árbol de altura $N - 1$ (el peor caso) para la implementación QuickUnion básica.

4. Implemente la estructura WeightedQuickUnion con compresión de caminos y compare experimentalmente su desempeño con QuickFind y QuickUnion básico para:

   - 1000 elementos con 5000 operaciones aleatorias
   - 10000 elementos con 50000 operaciones aleatorias

5. Considere el problema de determinar si un grafo no dirigido es conexo. Describa cómo usar Union-Find para resolver este problema eficientemente. ¿Cuál es la complejidad temporal de su solución?

6. **(Desafío)** Modifique la estructura WeightedQuickUnion para que también mantenga el tamaño de cada componente conexa de forma eficiente. Agregue una operación `componentSize(p)` que retorne el número de elementos en el componente que contiene a p.